chapter – 04

* Single thread vs Multithread process:

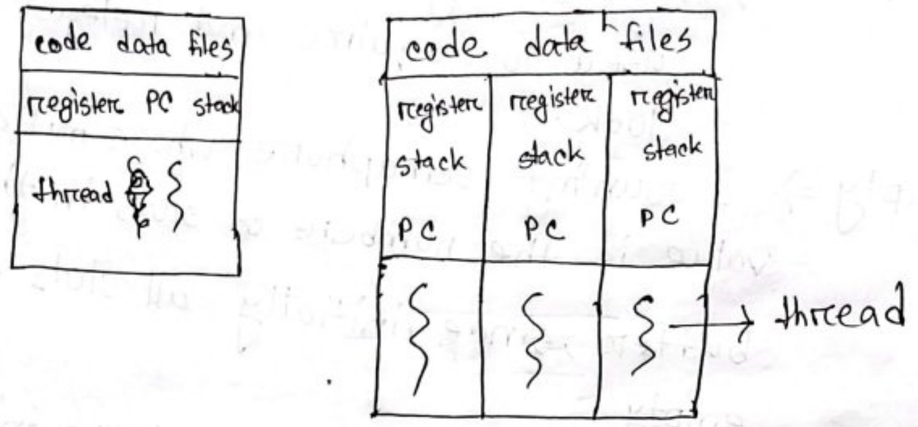


Fig: Single threaded and multithreaded process 2016-17 2(b)

Single thread	Multithread
1. It contains execution of instruction in a single sequence.	1. It contains execution of multiple parts of a program at the same time.
2. It executes one command at a time	2. It executes multiple command at a time.
3. It takes time	3. It saves time.

4.1] Three programming examples in which multithreading provides better performance:

1. A web server that services each request in a separate thread.
2. A parallelized application such as matrix multiplication where various parts of the matrix can be worked in parallel.
3. An interactive GUI program such as debugger where one thread is used to monitor input another thread represents the running application, and a third thread monitors performance

Thread: It refers to a single sequential flow of activities being executed in a process

4.2 Amdahl's Law: 16-17-2(a), 18-19-2(b,d)

It is a formula that identifies potential performance gain from adding additional computing cores to an application that has both serial and parallel components.

$$\text{speedup} \leq \frac{1}{S + \frac{1-S}{N}}$$

S = serial portion

N = Processing core

As given parallel component = 60%

∴ serial component = ~~60-40~~ 100-60

= 40%

= 0.4

a) For two processing core,

$$\text{speedup} \leq \frac{1}{0.4 + \frac{1-0.4}{2}}$$

$\leq$ 1.43 times.

b) For four processing core,

$$\text{speedup} \leq \frac{1}{s + \frac{1-s}{N}}$$

$$\leq \frac{1}{0.4 + \frac{1-0.4}{4}}$$

$$\leq 1.82 \text{ times.}$$

4.4| Difference between user level threads and kernel level threads : 2017-18-2(b)

User level thread	Kernel level thread
1. Use threads are implemented by user-level libraries.	1. Use threads are implemented by operating system.
2. Operating system doesn't recognize threads directly	2. Operating system recognize threads directly.
3. Implementation of user thread is easy.	3. Implement of user thread is complicated.

User level thread	Kernal level thread
4. Context switch time is less.	4. Context switch time is more.
5. Hardware support is not required.	5. Hardware support is required.
6. Can be created and manage quickly.	6. Takes time to be created and managed.
7. More portable.	7. Less portable.
8. Example: POSIX threads, mach C threads.	8. Example: Java threads, POSIX threads on Linux.

Creation and Management:

User level threads are created and managed by itself whereas kernel are created and managed by operating system. Though user level are typically faster but kernel level have more control over hardware.

Isolation: As user level is not is not isolated from each other it will lead problem when one thread crashes.

User level threads are better for applications that need to be fast and responsive. It can improve performance of the application.

Kernel-level are better for applications that need to be highly concurrent or need to access hardware resources directly.

Ultimately, the best type of thread to use depends on the specific application and its requirement

4'5] context switch between kernel threads typically require saving the value of CPU registers from thread being switched out and restoring CPU registers of new thread being scheduled. 2017-18-2(a) 18-19-2(b)

4'6] No additional resources are used when thread is created and they differ from those used when a process is created. Thread uses fewer resource than process Thread use first a method like process, and a method needs allocating Press control Block(PCB) PCB includes a memory map, a list of open files, and environment variables.

Kernel thread involves allocating a small data structure to hold a register ~~stack~~, set, stack and priority.

4.7] By binding an LWP (light-weight process) to a real time thread, you are ensuring that the thread will be able to run with a minimal delay once it is scheduled.

4.8] Two programming examples in which multithreading does not provide better performance than single threading:

1. Any kind of sequential program is not good condidate to be threaded. Tax return.
2. Shell program such as C-shell or Korn shell.

4.9] Circumstances in which multithread using kernel method ~~is~~ to provide better performance are:

⇒ The application is CPU intensive. Multithread can improve performance by allowing CPU to switch between threads that are waiting for resources.

⇒ Multithread can be used in a waiting for I/O and improve performance by allowing the CPU to switch other threads that are not waiting for I/O.

⇒ Multithread solution can be used in system resource allowing operating system to better manage the resources.

4'15
4'10] ⇒ Using a separate thread to generate a thumbnail for each photo in a collection

⇒ Task parallelism. Each thread is running the same code but different data.

⇒ Transposing a matrix in parallel.

⇒ Data parallelism. Each thread is performing same task on different subset of data.

⇒ A network application where one thread reads from the network and another

writes to the network.

⇒ Task parallelism. Each thread is performing different task on same data.

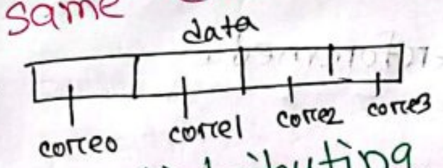
⇒ The fork-join array summation application

⇒ Data parallelism. Each thread is performing same task but different data.

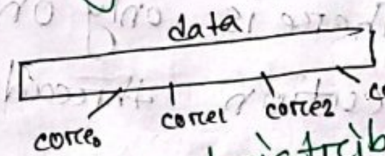
⇒ The Grand Central Dispatch system.

⇒ Task parallelism: Each thread performing different task on the same data

*Data Parallelism: 2018-19 - 2(a)



Focuses on distributing subsets of same data on different data across multiple computing core and performing the same task.



* Task Parallelism: 2018-19 - 2(a)

Focuses on distributing different tasks across multiple core and performing unique operation.

*Difference between Data parallelism and task parallelism:

Data parallelism	Task parallelism
1. Same task are performed on different data.	1. Different task are performed on same or different data.
2. Synchronous computation is performed.	2. Asynchronous computation is performed.
3. Speedup is ~~less~~ more	3. Speedup is less.
4. There is only one execution thread.	4. There is different execution for each thread.
5. Amount is parallelization is proportional to input size.	5. Amount is parallelization is proportional to independent task.
6. It is designed for optimum ~~load~~ load balance.	6. ~~It is~~ Its load balance depends on availability of the hardware.

Q.20] ⇒ a) When number of kernel threads is less than number of processors, then some of the processors would remain idle. since scheduler maps kernal threads to processor not user threads to processor

b) When the number of kernel threads is exactly equal to the number of processor, then it is possible that all of the processors might be utilized simultaneously.

c) When the number of kernel threads are more than processor, a blocked kernel thread could be swapped out in favor of another kernel thread.

অনেক কাজ একসাথে হওয়া

* Show the concurrent execution on a single-core system and parallel multicore system. 2018-19 - 2(d)

Soln:

Single-core system	Multicore system
1. Single core system achieve ~~ga~~apparent concurrency.	1. Multicore system can achieve true concurrency.
2. Perform tasks slowly.	2. Perform task fastter.
3. Managing is easy.	3. Managing is complex.
4. Only one task runs at a time	4. Multiple tasks run simultaneously

* Describe about different multithreading model. 2015-16 - 8(c)

Soln: There are three model. They are:

1. Many to one model:

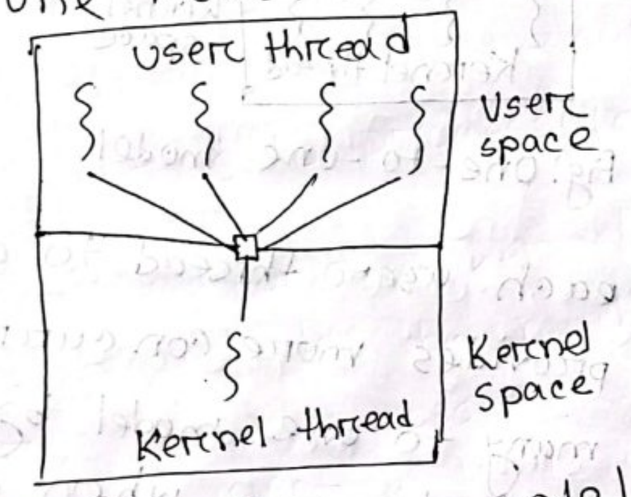


Fig: many-to one model.

It maps many user level threads to one kernel thread. Thread management is done by thread library in user space. The entire process will block if a thread makes a blocking system call; One thread can access the Kernel at a time, multiple threads are unable to run in parallel on multicore system.

2. One-to-one model:

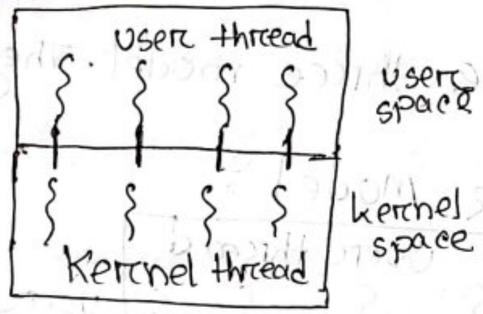


Fig: One-to-one model

It maps each user thread to a kernel thread. It provides more concurrency than the many to one model by allowing another thread to run when a thread makes blocking system.
The only drawback is that creating a user thread requirings creating corresponding kernel thread, and large number of kernel thread may burden the performance of a system.

3. Many - to - Many model

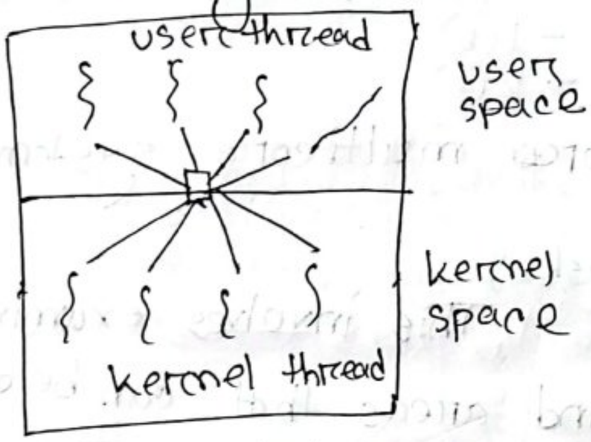


Fig - many to many

It multiplexes many user level threads to a smaller or equal kernel threads. The number of kernel threads may be specific to either a particular application or particular machine.

*Discuss the challenges in multicore system.

Soln: 15-16 - 1(b)
16-17 - 2(b)

Challenge for multicore system:

1. Identifying tasks: This involves examining applications to find areas that can be divided into separate, concurrent tasks. Tasks are independent of one another and thus can run in parallel on individual cores.

2. Balance: While identifying tasks that can run in parallel, programmers must also ensure that the tasks perform equal work of equal value. Sometimes, task may not contribute equa value to overall process. Using separate execution core to run that task may not be worth the cost.

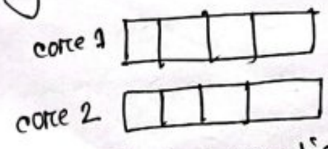


Fig: Parallel execution on a multicore system.

3. Data splitting:
As applications are divided into separate tasks, the data accessed and manipulated by the tasks must be divided to run on separate cores.

4. Data dependency:
The data accessed by the tasks must be examined for dependencies between two or more tasks. When one task depends on data from another, programmer must ensure that the execution of tasks is synchronized to acomodate the data dependency.

5. Testing and debugging:
When a program is running on multiple cores, many different execution paths are possible. Testing and debugging such concurrent programs more difficult than single threaded applications.

* Briefly describe various methods of Implicit threading.

Soln:

Various methods of implicit threading

2015-16 - 4 (16)

1. Thread Pools: → necessity and benefits

In this ~~mode~~ method a collection of pre-created threads is maintained. When a task arrives, the system assigns it to an available thread from the pool than creating a new thread.

2. OpenMP (Open Multi-Processing):

An API is used in C, C++ for parallel programming. It allows programmers to specify parallel regions in the code.

3. Grand Central Dispatch (GCD):

It is a technology that provides automatic thread management

4. Fork-Join Model

It allows program to be divided into smaller tasks than can be executed in parallel.

CamScanner

Chapter - 5

5.1 CPU scheduling:

It is the process by which an operating system decides the order in which processors or thread should be executed by CPU.

5.1] Given n process to be scheduled on one processor. ~~There~~ Possible schedules are:

n!

5.2] Difference between preemptive and non-preemptive scheduling:

Preemptive scheduling	Non-preemptive scheduling
1. It is a method that may be used when a process switches from running state to a ready state.	1. It is a method that is used when a process terminates or switches from a running to a waiting.
2. Its process may be paused in the middle of execution.	2. It does not interrupt in the middle.

Preemptive scheduling	Non-preemptive scheduling
3. It is flexible.	3. It is rigid.
4. It is cost associated.	4. It does not cost associated.
5. It has overheads.	5. It doesn't have overhead.
6. It affects the design of the operating system.	6. It doesn't affects the design of the operating Kernel
7. Its CPU utilization is very high.	7. Its CPU utilization is very low.
8. Example: Round Robin and Shortest Remaining Time First	8. Example: FCFS and SJF are examples of non preemptive scheduling.

*Scheduling Algorithms:

⇒ First-Come, First-Served scheduling: (FCFS)

⇒ The process that requests the CPU first is allocated the CPU first.

⇒ It is easily managed with a FIFO queue.

⇒ When a process enters the ready queue, its PCB is linked onto the tail of the queue.

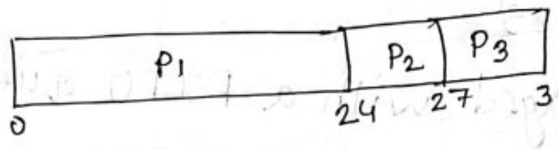
⇒ When the CPU is free it is allocated to the process at the head of the queue. The running process is then removed from the queue.

Example:

Process	Burst time (milliseconds)
P_1	24
P_2	3
P_3	3

Here set of processes arrive at time 0 with length of CPU burst given in miliseconds:

- If the process arrive in the order P_1, P_2, P_3 and served in FCFS order, we get following Gantt chart:



The waiting time is 0 miliseconds for P_1, 24 ms for P_2, 27 ms for P_3.

∴ Average time $= \frac{(0 + 27 + 27)}{3}$

$= 17$ ms.

⇒ FCFS is non preemptive.

Advantages

1. It is simple.
2. Every process is treated equally on its arrival of time
3. It has minimal scheduling overhead.
4. It is easy to model.
5. Works well in environments where processes have long burst.

Disadvantages

1. Poor CPU utilization
2. Long waiting times.
3. Lack of prioritization
4. Non-preemptive nature.

*Shortest-Job-First Scheduling : (SJF)

⇒ This algorithm associates with each process the length of the process's next CPU burst.

⇒ When CPU is available, it is assigned to the process that has the smallest next CPU burst.

⇒ If the next CPU burst of two process are same, FCFS scheduling is used to break the tie.

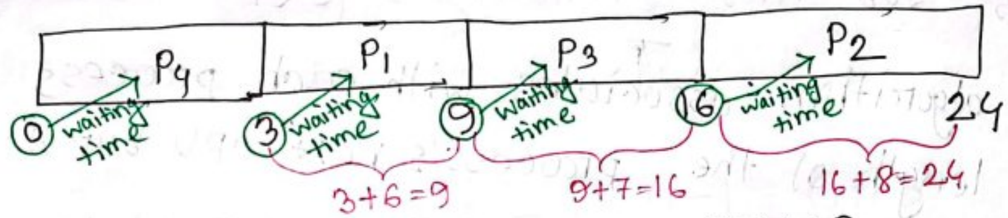
⇒ It can be called shortest-next-CPU-burst algorithm because scheduling depends on length of the next CPU burst of a process.

Example: non-preemptive

the amount of CPU time the process require to complete execution

Process	Burst time (execution time)
P_1	6
P_2	8
P_3	7
P_4	3

Using SJF scheduling, we would schedule these process according to the Gantt chart:



~~Average waiting time = 3+16+9~~

The waiting time for P_1 = 3 ms

" " " " P_2 = 16 "

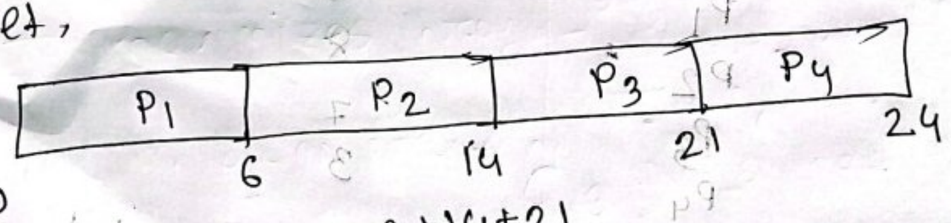
" " " " P_3 = 9 "

" " " " P_4 = 0 "

∴ Average waiting time = $\frac{3+16+9+0}{4}$

$= \frac{28}{4}$ = 7 ms

if we use FCFS scheduling then we get,



Average time = $\frac{6+14+21}{4}$

= 10.25 ms.

So, we can say that SJF scheduling is probably optimal because it gives the minimal average time.

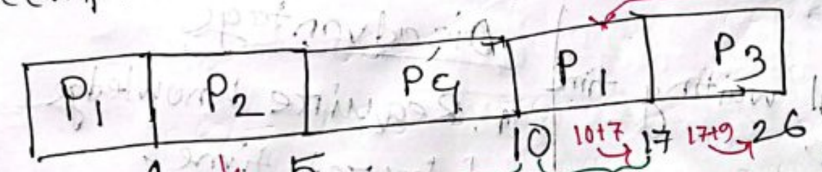
P_1 P_2 (8, 17)

Example: preemptive scheduling → the process that has the next smallest remaining CPU burst time

Process	Arrival time	Burst time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

After 1 sec P1 (7)

Preemptive SJF schedule is:



শুরুতে arrival time=0 তে P_1 ছিল তাই আসছে

P_2 আসছে কারণ P_2 এর burst time P_1 এর থেকে কম using SJF

P_3 and P_4 burst time compare হবে যেহেতু P_4 কম তাই P_4 আসবে

*Waiting time = Total waiting time − No. ms Process executed − Arrival time..

*Preemptive SJF → Shortest-Remaining time first scheduling.

waiting time for $P_1 = 10 - 1 - 0 = 9$ ms

" " " $P_2 = 1 - 0 - 1 = 0$ ms

" " " $P_3 = 17 - 0 - 2 = 15$ ms.

" " " $P_4 = 5 - 0 - 3 = 2$ ms

∴ Average waiting time $= \frac{9+0+15+2}{4}$

$= 6.5$ ms.

Advantage :

1. Reduce overall waiting time for processes.
2. Ensures a certain degree of fairness.
3. It maximizes through put.
4. Well in batch processing system.

Disadvantage

1. Require knowledge of burst time.
2. Long process can suffer if short processes keep arriving.
3. Not suitable for real time system.

*Round – Robin scheduling:

⇒ Similar to FCFS but preemption is added to enable the system to switch between processes.

⇒ A small unit of time is defined called time quantum.

⇒ Time quantum is generally from 10 to 100 milliseconds in length. The ready queue is treated as circle queue.

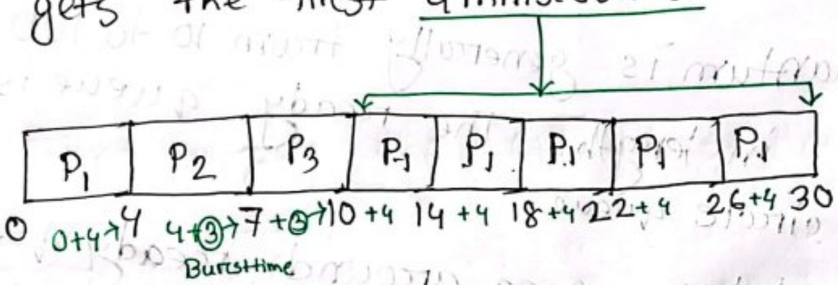
⇒ CPU scheduler goes around ready queue, allocating CPU to each process for time interval up to 1 time quantum.

⇒ CPU scheduler picks the first ready queue, sets a timer to interrupt after 1 time quantum and dispatches process.

Example:

Process	Burst time
P_1	24
P_2	3
P_3	3

Consider the process the arrrive time 0.
If we use a time quantum of 4ms. then
P_1 gets the first 4miliseconds



Since it requirres another 20ms. It is preempted after first quantum and the CPU is given to next process in the queue. Process P_2, P_2 does not need 4ms. so it quits before its time quantum expirres. The CPU is given to next process, process P_3. Once each process has received 1 time quantum, CPU returns to process P_1 for an additional time quantum. ~~The resulting RR schedule is:~~

Waiting time for P_1 = 10 - 4

= 6 ms

" " " P_2 = 4 ms

" " " P_3 = 7 ms

" " " P_4 = 1

∴ Average waiting time = $\frac{6+4+7}{3}$

= 5.67 ms.

~~Exa~~

Advantage:

1) It is premptive
2. Idea for time sharing system
3. Prevent starvation.
4. Ensures predictable response time.
5. Works well for uniform process

Disadvantage:

1. High context-switching overhead
2. sensitive to the choice of time quantum
3. Inefficient for short process.
4. Can result in long turnaround times for certain workloads

*Priority Scheduling:

⇒ SJF is a special case of general priority scheduling algorithm.

⇒ A priority algorithm is associated with each process. and CPU is allocated to the process with high priority.

⇒ The larger the CPU Burst the lower the priority.

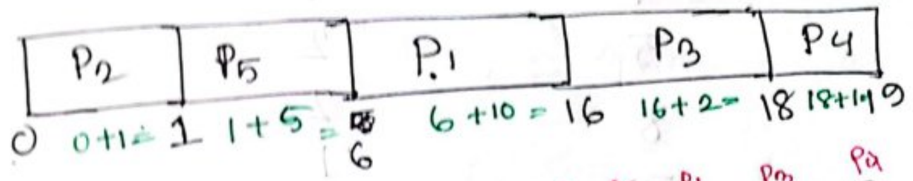
Non preemptive priority will simply put the new process at the head of the ready queue.

⇒ Examples:

Process	Burst	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Here low number represent high priority.
Using priority scheduling, we would schedule

these processes according to the following Grantt chart:



Average waiting time = $\frac{0+1+6+16+18}{5}$ (P2 P5 P1 P3 P4)

$= 8.2$ ms.

Advantage :

1) Efficient for critical task
2) Flexible.
3) Reduce waiting time for high priority tasks.
4. suitable for realtime system.
5. Improve responsiveness.

Disadvantage

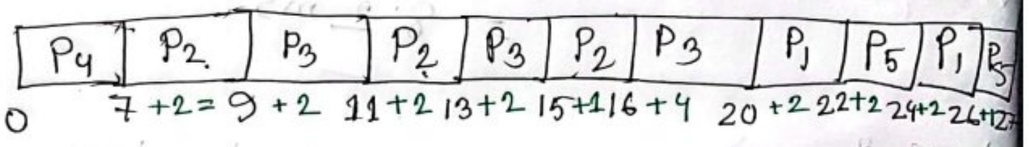
1. Risk of starvation.
2. Complexity in managing priorities
3. Priority inversion
4. Unfairness to lower priority task.

* A preemptive priority scheduling will preempt the CPU if the priority of the newly arrived process is higher than the priority of currently running process.

Example:

Process	Burst	Priority
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

time quantam = 2 ms.



P_4 has the highest priority, so it will run to completion. Process P_2 and P_3 have next-highest priority and they will execute in round-robin execution. As time quantum = 2ms.

At first P_2 will go. = 7+2 = 9 ms

then P_3 " " = 9+2 = 11 "

" P_2 " " = 11+2 = 13 "

" P_3 " " = 13+2 = 15

" P_2 " " = 15+1 = 16 → কারণ P_2 5ms কাজ করবে তাই প্রথম 2 বার 2ms কাজ করার পর 1ms বাকি

then P_3 = 16+4 = 20

↳ P_3 at first 2ms করে কাজ করবে then ~~Priority~~ burst P_2, P_3 এর কাজ শেষ হলে P_3 এর বাকি যত time কাজ বাকি থাকবে করবে. P_3 এর Last এ 4ms বাকি ছিল.

Now, P_1 and P_5 remain and they have equal priority. As FCFS P_1 will first enter then it will take 2ms after that P_5 will enter, it will take 2ms. They will execute in round robin order until they complete.

*Multilevel Queue Scheduling:

⇒ It ~~is~~ works well when priority scheduling is combined with round-robin.

⇒ If there are multiple processes in the highes priority queue, they are executed in round-robin order.

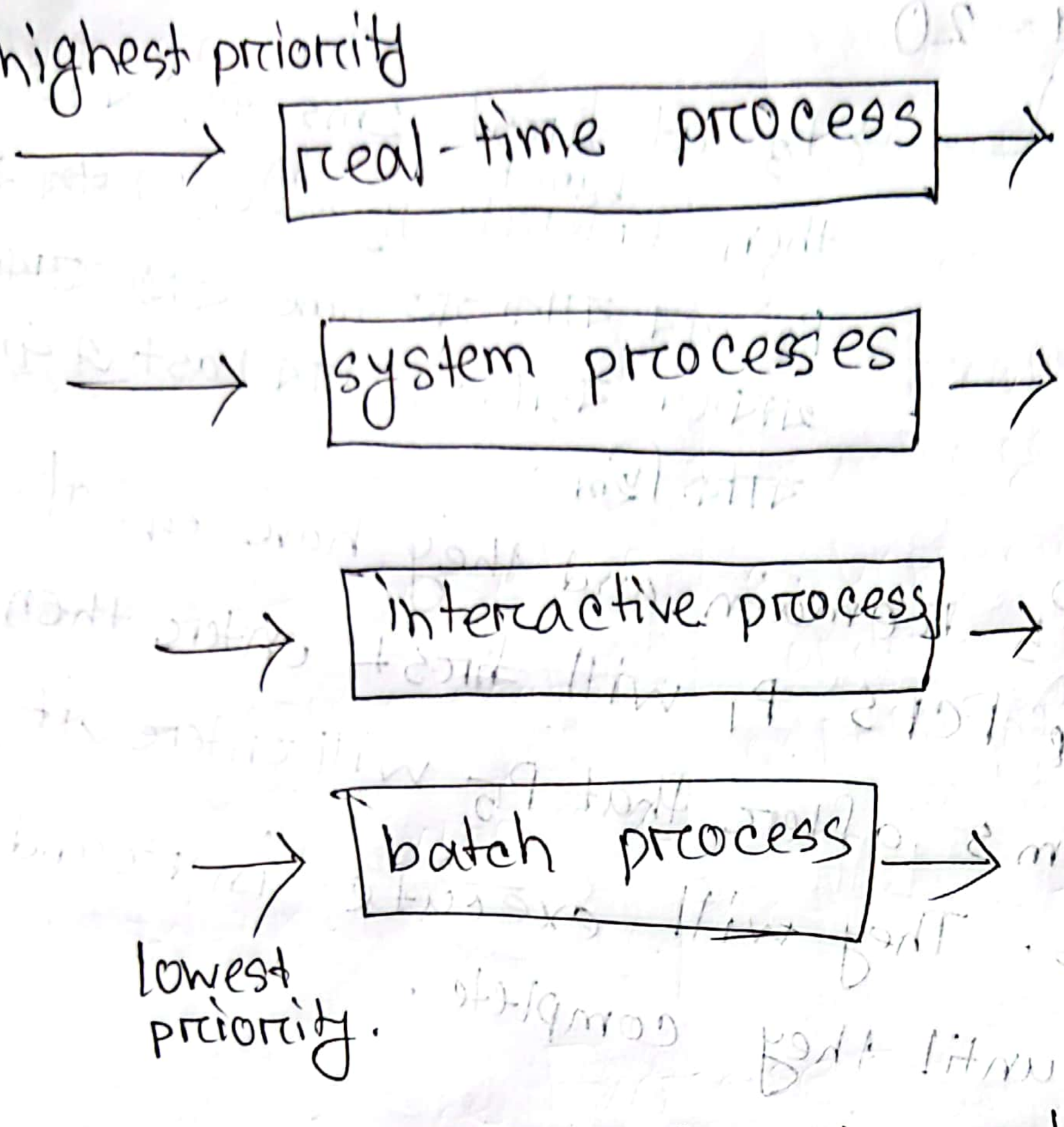


Fig: Multilevel Queueing scheduling.

⇒ Each queue has absolute priority over lower priority queues.

⇒ No process could run unless the queues of its previous process are all empty.

Advantage:

1. Efficient for diverse workloads
2. Specialized handling of different process type
3. Customizable.
4. Good separation of processes.

Disadvantage

1. Rigid queue structure.
2. Potential inefficiencies in CPU.
3. Complexity in management
4. Unfair delay for low priority process.

*Multilevel Feedback Queue Scheduling

It allows a process to move between queues. The idea is to separate process according to the characteristic of their CPU bursts.

⇒ A process that waits too long in a lower priority queue may be moved to a higher-priority queue.

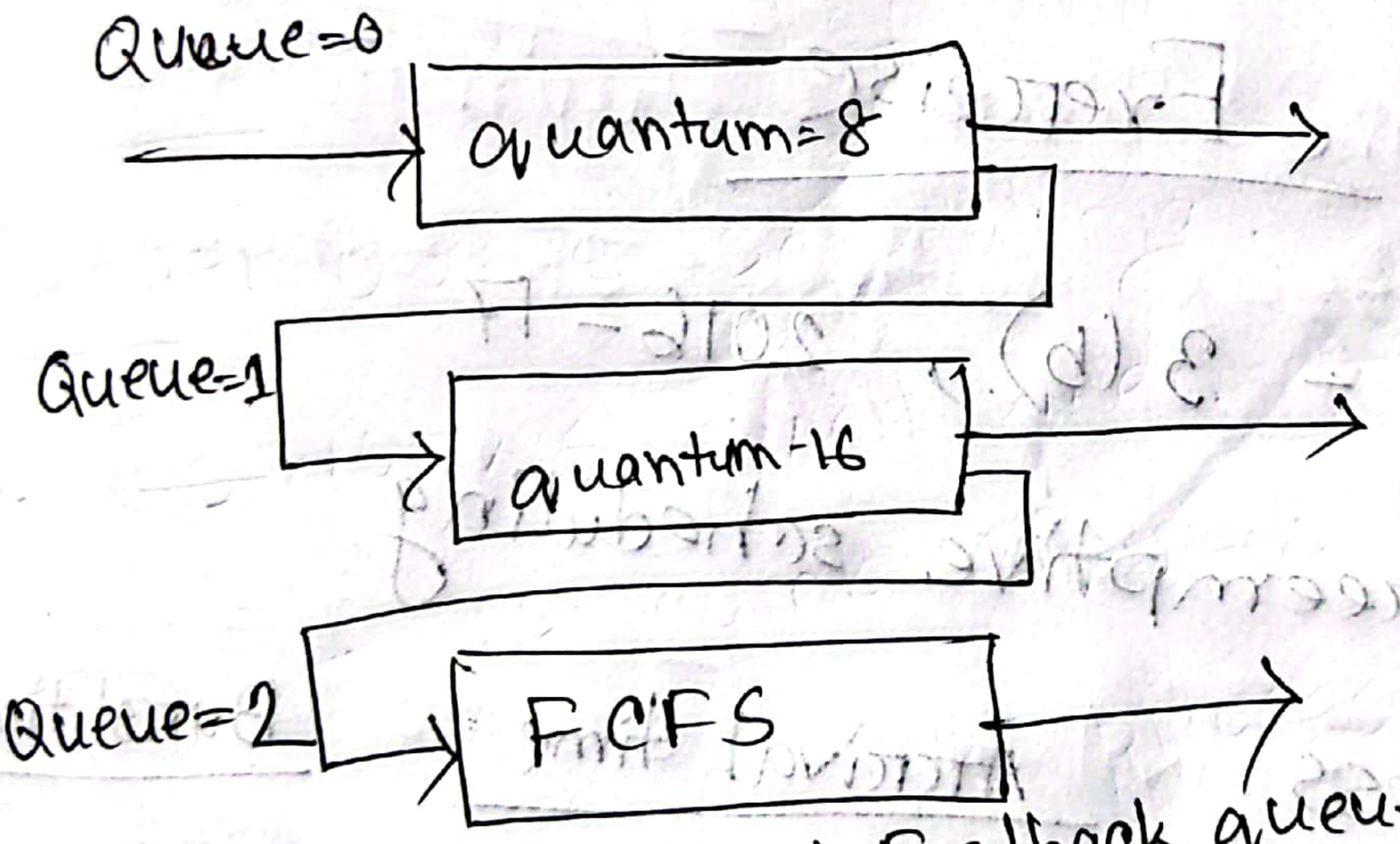


Fig: Multilevel Feedback queues

An entering process is put in queue 0. A process in queue 0 is given a time quantum of 8 miliseconds. If it does not finish within this time, it is moved to the tail of queue 1. If queue 0 is empty, the process at the head of queue 1 is given quantum 16 ms. If it is not completed then is put into queue 2

Advantage

1. Dynamic adjustment of process priority
2. Better response time for interactive tasks.
3. Prevents starvation.
4. Fair CPU allocation.
5. Work well with mix workloads.

Disadvantage

1. Complex to implement
2. Higher overhead due to frequent context switching.
3. Unpredictable execution order.
4. Difficult to tune.

Exercise

5.3| 2018-19 - 3 (b), 2016-17

using non preemptive scheduling

Process	Arrival time	Burst time
P_1	0.0	8
P_2	0.4	4
P_3	1.0	1

Now, P_2 ~~co~~ finishes at 12ms. P_3 runs 1ms.

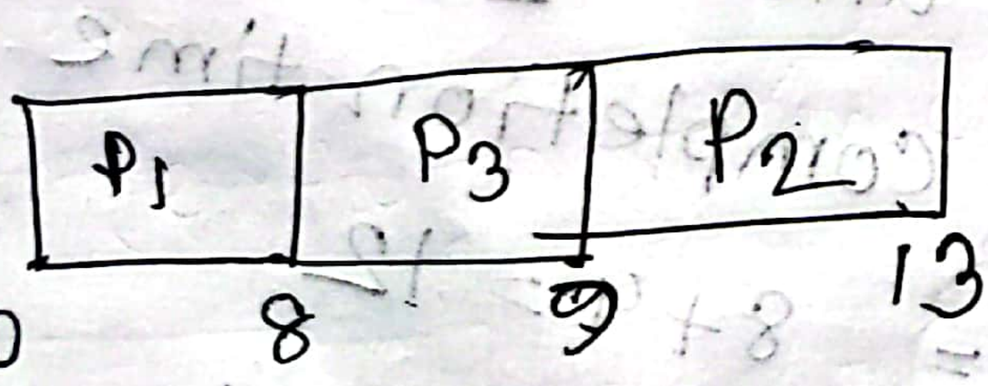
∴ Completion time = 12+1 = 13ms

∴ Turnaround time for P_3 = 13 − 1
= 12

∴ Average turn around time = $\frac{8+11.6+12}{3}$
= 10.53 (Ans)

ii) What is the average turnaround time of these processes with SJF scheduling algorithm.

As P_1 has arrival time 0.0 .so it starts first after P_1 finishes P_3 will run next as it has short burst time. Then P_2 runs.



Turnaround for $P_1 = 8 - 0 = 8$

Turn around for P_2:

completion time $= 8 + 1 = 9$

Turnaround time for $P_2 = 9 - 1 = 8$

Turnaround for P_3:

Completion time $= 9 + 4 = 13$

Turnaround for $P_3 = 13 - 0.4 = 12.6$

Average turnaround $= \frac{8 + 8 + 12.6}{3}$

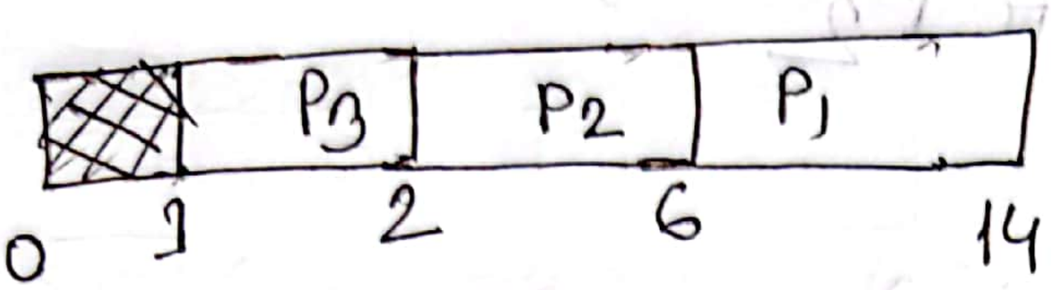
$= 9.53$ (Ans)

iii) The SJF algorithm is supposed to improve performance, but notice that we chose to run process P1 at time 0 because we did not know that two shorter processes would arrive soon. Compute what the turnaround time will be if CPU is idle for first 1 unit and then SJF is used. Remember that P_1 and P_2 will wait during this idle time. so their waiting is increased. This algorithm is called future-knowledge algorithm.

Soln:

The CPU is idle from 0 to 1;

At time 1 P_3, P_1, P_2 are in queue, since P_3 has shortest burst time, it run first.



Now, P3:

completion time = 2

turnaround time = 2 - 1 = 1

P_2:

completion time = 12 + 4 = 6

turnaround time = 6 - 0.4 = 5.6

P_1:

Completion time = 6 + 8 = 14

turnaround time = 14 - 0 = 14

Average turnaround time = $\frac{1 + 5.6 + 14}{3}$

= 6.87 (Ans)

21-22

5.4]

Process	Burst time	Priority
P_1	2	2
P_2	1	1
P_3	8	4
P_4	4	2
P_5	5	3

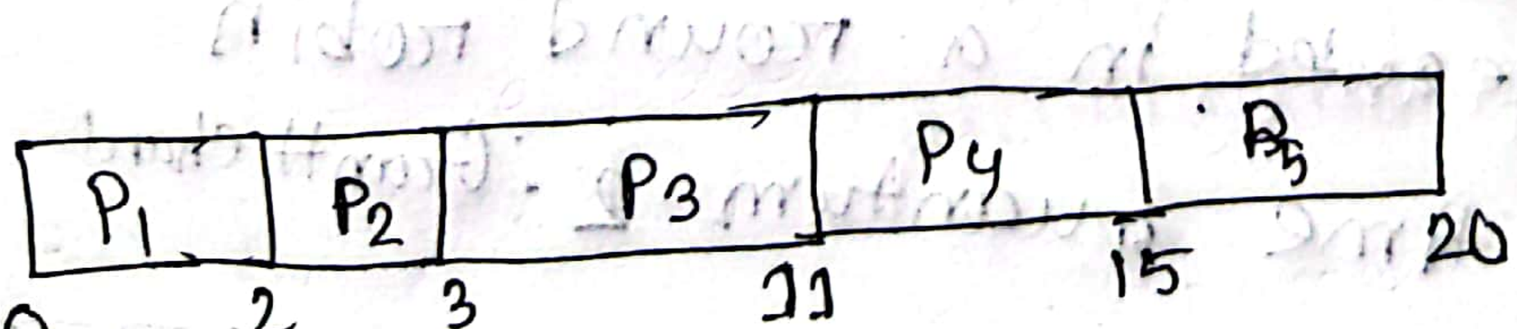
higher number higher priority.

Draw four Gantt chart that illustrates the execution of these processes using the following scheduling algorithm: FCFS, SJF, nonpreemptive priority and RR (quantum = 2)

Soln:

⇒ FCFS :

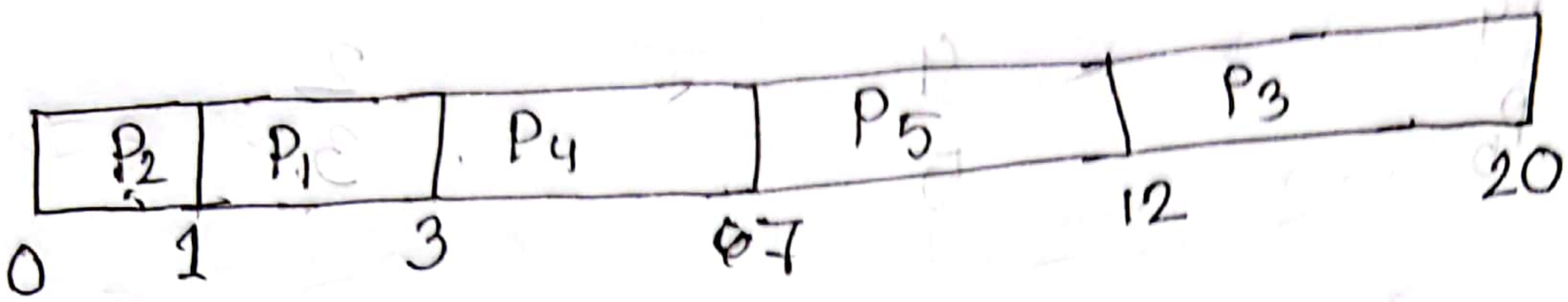
since all processes arrive at time 0, FCFS run them when they arrive. Gantt chart:



⇒ SJF:

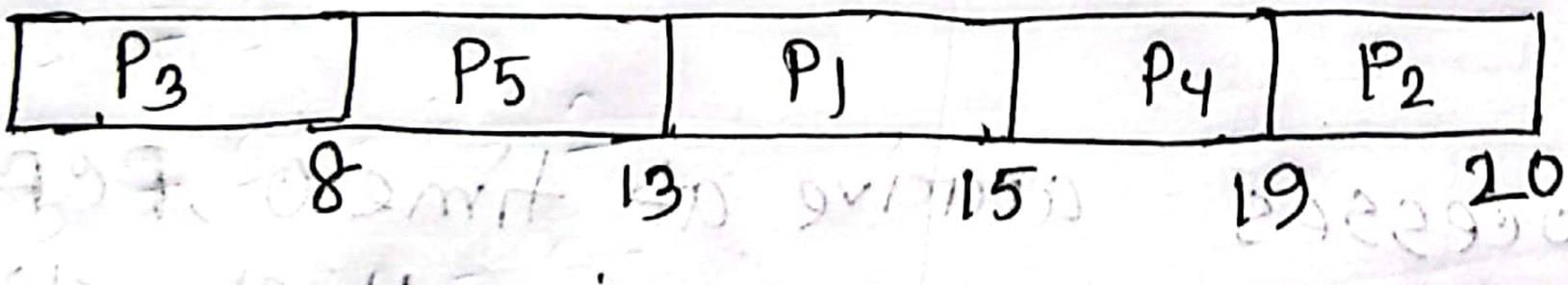
Processes are scheduled in the order of their burst time, from shortest to longest

Grantt Chart:



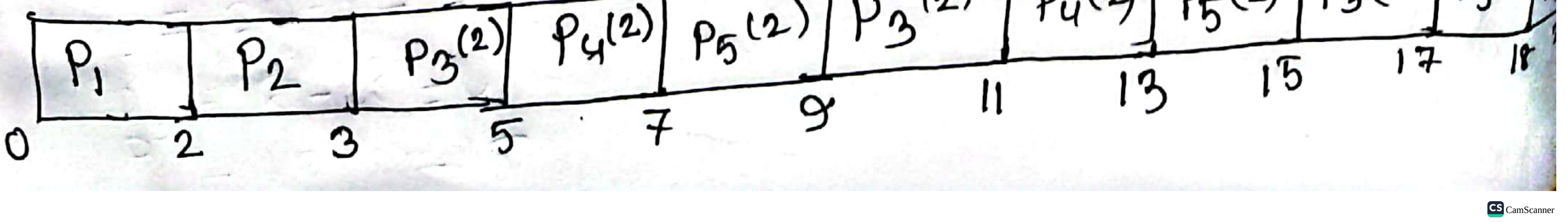
⇒ Non preemptive Priority scheduling (Larger number = higher priority):

Processes are scheduled based on their priority. If priority are same FCFS is used. Grantt chart:



⇒ Round Robin with quantum = 2:

Process are executed in a round robin fashion with a time quantum 2. Grantt chart



i) what is the turnaround time of each process for each scheduling in part-(i)

Soln:

FCFS Turnaround time = Completion time - Arrival time

Turnaround time:

$P_1 = 2 - 0 = 2$

$P_2 = 3 - 0 = 3$

$P_3 = 11 - 0 = 11$

$P_4 = 15 - 0 = 15$

$P_5 = 20 - 0 = 20$

⇒ Round Robin turnaround time:

$P_1 = 2 - 0 = 2$

$P_2 = 3 - 0 = 3$

$P_3 = 20 - 0 = 20$

$P_4 = 13 - 0 = 13$

$P_5 = 18 - 0 = 18$

⇒ SJF Turnaround time:

$P_1 = 3 - 0 = 3$

$P_2 = 1 - 0 = 1$

$P_3 = 20 - 0 = 20$

$P_4 = 7 - 0 = 7$

$P_5 = 12 - 0 = 12$

⇒ Non preemptive turnaround time:

$P_1 = 15 - 0 = 15$

$P_2 = 20 - 0 = 20$

$P_3 = 8 - 0 = 8$

$P_4 = 19 - 0 = 19$

$P_5 = 13 - 0 = 13$

iii) What is the waiting time of each proc for each of scheduling algorithm:

Soln:

Waiting time = Turnaround time - Burst time

⇒ FCFS:

$P_1 = 2 - 2 = 0$

$P_2 = 3 - 1 = 2$

$P_3 = 11 - 8 = 3$

$P_4 = 15 - 4 = 11$

$P_5 = 20 - 5 = 15$

⇒ SJF:

$P_1 = 3 - 2 = 1$

$P_2 = 1 - 1 = 0$

$P_3 = 20 - 8 = 12$

$P_4 = 7 - 4 = 3$

$P_5 = 12 - 5 = 7$

⇒ Non preemptive priority:

$P_1 = 15 - 2 = 13$

$P_2 = 20 - 1 = 19$

$P_3 = 8 - 8 = 0$

$P_4 = 19 - 4 = 15$

$P_5 = 13 - 5 = 8$

⇒ Round Robin:

$P_1 = 2 - 2 = 0$

$P_2 = 3 - 1 = 2$

$P_3 = 20 - 8 = 12$

$P_4 = 15 - 4 = 11$

$P_5 = 18 - 5 = 13$

v) Which of the algorithms result in the minimum averag waiting time (over all process)

=> FCFS Average time = $\frac{0+2+3+11+15}{5}$

$= 6.2$

=> SJF Average time = $\frac{1+0+12+3+7}{5}$

$= 4.6$

=> Priority Average time = $\frac{13+19+0+15+8}{5}$

$= 11$

=> Round Robin " " = $\frac{0+2+12+11+13}{5}$

$= 7.6$

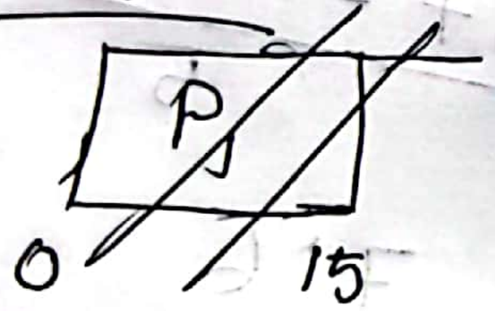
So, we can say that the SJF (shortest Job-First) scheduling algorithm results in the minimum average waiting time of 4.6ms.

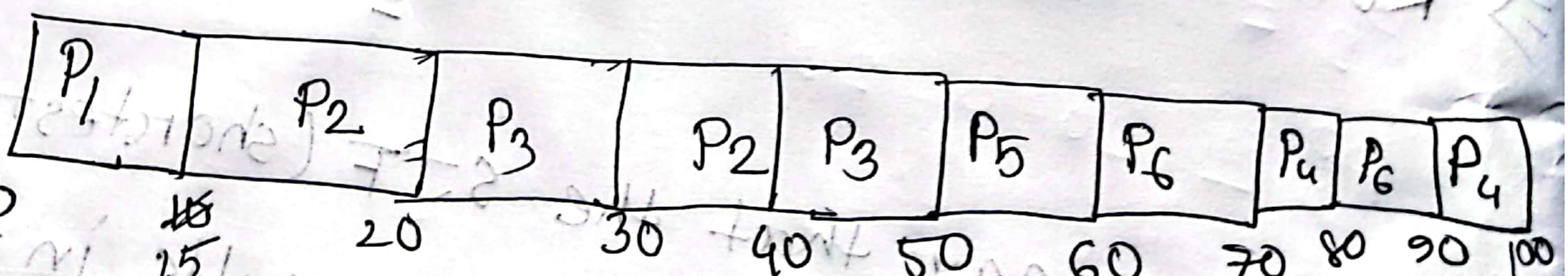
5.5). 2017-18 - 4(c)

Process	Priority	Burst time	Arrival time
P_1	8	15	0
P_2	3	20	0
P_3	4	20	20
P_4	4	20	25
P_5	5	5	45
P_6	5	15	55

i) Show the scheduling oreder of the processes using Gantt chart.

Soln:





ii) What is the turnaround time for each process.

Soln:

Turnaround time = Completion − Arrival time

Process	completion time	Arrival time	Turnaround
P_1	10	0	10
P_2	40	0	40
P_3	50	20	30
P_4	100	25	75
P_5	60	45	15
P_6	90	55	35

iii) ~~wa~~ What is the waiting time for each process?

Soln: Waiting time = Turnaround − Burst time

Process	Turnaround	Burst time	Waiting time
P_1	~~10~~15	15	~~5~~0
P_2	40	20	20
P_3	30	20	10
P_4	75	20	55
P_5	15	5	10
P_6	35	15	20

* CPU utilization = $\frac{\text{Total Busy time}}{\text{total time}} \times 100$

$= \frac{100}{100} \times 100$

$\leq 100\%$